

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Mendoza

Tecnicatura Universitaria en Programación (TUP)

Asignatura: Organización Empresarial

Docente: Prof. Francisco Alejandro Varberde Thompson

Ciclo Lectivo: 2026 | **Semestre:** 1°

TRABAJO PRÁCTICO N° 1: INGENIERÍA DE PROMPTS Y GIT/GITHUB

1. Instrucciones Generales

El objetivo de este trabajo práctico es evaluar la comprensión y aplicación práctica de los conceptos de calibración, restricciones y reglas de sintaxis en la ingeniería de prompts, así como el uso básico de Git y GitHub para la entrega de trabajos.

Formato y Condiciones de Entrega:

1.

Desarrollo: Todo el desarrollo de las consignas debe realizarse inicialmente en un documento de texto (Word, Google Docs, etc.).

Exportación: Una vez finalizado, el documento debe ser exportado obligatoriamente a formato **PDF**.

Repositorio: El archivo PDF deberá subirse a un repositorio **público** alojado en

GitHub.

1.

Entrega: La entrega formal del trabajo consistirá **únicamente en el enlace (link) del repositorio de GitHub**, el cual debe contener el archivo PDF de forma visible y accesible.

1. Formato Requerido para las Actividades

Para que la resolución de cada consigna sea considerada válida, los estudiantes deberán incluir la siguiente estructura por cada punto evaluado:

1.

El Prompt: El texto exacto del mensaje que enviaron al modelo de lenguaje.

1.

La Evidencia: Una captura de pantalla clara y legible de la respuesta generada por el modelo.

1. Consignas a Desarrollar

Actividad 1: Calibración del Modelo (Temperatura)

Demostrar el control sobre el nivel de determinismo y creatividad del modelo:

1.

1.a. Precisión: Realizar un prompt diseñado para obtener una respuesta de alta precisión y objetividad (baja temperatura).

1.b. Creatividad: Realizar un segundo prompt buscando una respuesta altamente creativa o divergente (alta temperatura).

Actividad 2: Restricción de Longitud (Max Length)

Controlar la cantidad de tokens generados en la salida:

1.

Redactar un prompt donde se evidencie una restricción explícita sobre el uso de tokens o la longitud máxima permitida para la respuesta.

Actividad 3: Secuencias de Parada (Stop Sequences)

Interrumpir la generación de texto de manera controlada:

1.

Desarrollar un prompt que ponga en uso la restricción de *Stop Sequences*, logrando que el modelo detenga su respuesta en un punto específico predefinido.

Actividad 4: Control de Repetición y Novedad (Penalizaciones)

Ajustar el comportamiento del modelo frente a la repetición de términos o temas:

1.

4.a. Frequency Penalty: Realizar un prompt donde se aplique la penalización de frecuencia.

4.b. Presence Penalty: Realizar un prompt donde se aplique la penalización de presencia.

Actividad 5: Reglas de Sintaxis Avanzada

Aplicar múltiples buenas prácticas de estructuración de prompts en un solo bloque de instrucciones:

1.

Diseñar un prompt complejo en el que se apliquen simultáneamente por lo

menos **3 de las siguientes 4 reglas de sintaxis**:

- .1.**Prioridad Posicional:** Ubicar las instrucciones críticas en las posiciones de mayor peso.
- .2.**Aislamiento de Variable:** Separar claramente el contexto o los datos de las instrucciones mediante delimitadores.

1.

- .1. **Especificidad:** Ser extremadamente detallado y preciso sobre el formato, tono y contenido.
- .2. **Evitar Respuestas Negativas:** Formular las restricciones en positivo en lugar de indicar qué no hacer.

3.1.a-

Mensaje enviado: temperatura 0.1, Explica el funcionamiento de una API.

Respuesta:

temperatura 0.1, Explica el funcionamiento de una API.

Cómo funciona una API

Una **API (Interfaz de Programación de Aplicaciones)** es un intermediario que permite que diferentes sistemas o aplicaciones se comuniquen entre sí. Piénsalo como un "mesero" en un restaurante: tú (cliente) pides, el mesero transmite al chef, y trae la respuesta.

Funcionamiento básico

Tu aplicación → API → Servidor/Base de datos → API → Tu aplicación
(solicitud) (procesa) (respuesta)

Los 4 pasos principales

1. Solicitud (Request)

Tu aplicación envía una petición a la API con:

- **Método:** qué acción hacer (GET = obtener, POST = crear, PUT = actualizar, DELETE = eliminar)
- **URL/Endpoint:** dónde enviar la solicitud
- **Parámetros:** datos específicos (ej: id del cliente, filtros)
- **Autenticación:** credenciales si es necesario (token, API key)



2. Procesamiento

La API recibe la solicitud y:

- Valida que sea correcta
- Verifica permisos
- Busca/modifica datos en la base de datos
- Aplica lógica empresarial

3. Respuesta (Response)

La API devuelve:

- **Código de estado:** 200 (éxito), 404 (no encontrado), 500 (error servidor)
- **Datos:** en formato JSON, XML o similar
- **Metadatos:** información adicional

4. Integración

Tu aplicación procesa la respuesta y la muestra al usuario.

3.1.b-

Mensaje enviado: temperatura 1.0, Explica el funcionamiento de una API

Respuesta:

¿Qué es una API? (Versión menos aburrida)

Una API es básicamente un **contrato entre amigos**: tú le pides algo a alguien de una forma específica, y esa persona te responde de una forma predecible. Sin necesidad de saber cómo lo hace.

La analogía del restaurante (clásica pero funciona)

Imagina que quieres comida. No entras a la cocina gritando al chef. En su lugar:

- 🍽️ Das tu **pedido** al mesero (request)
- 🍷 El mesero lo lleva a la cocina (API procesa)
- 👨‍🍳 El chef prepara (backend hace su magia)
- 🍴 El mesero trae tu comida (response)

Sin API, tendrías que entrar a la cocina, hablar directo con el chef, moverte por los trastos... caos total.

Los "verbos" que hablan las APIs

Verbo	Qué hace	Ejemplo
GET	Pedir información	"Dame los datos del usuario 123"
POST	Crear algo nuevo	"Registra este nuevo usuario"
PUT	Actualizar algo	"Cambia la contraseña del usuario"
DELETE	Borrar algo	"Elimina este registro"

El flujo real

Tu app: "Oye API, dame el clima de Buenos Aires"

↓

API: *valida que sea una solicitud válida*

↓

API: *consulta la base de datos de clima*

↓

API: "Aquí está: 25°C, soleado"

↓

Tu app: *muestra "25°C, soleado" al usuario*



Lo interesante: por qué existen

Las APIs son la razón por la que:

- 🏦 Tu app del banco se conecta con los servidores del banco
- 🗺️ Google Maps sabe dónde estás
- 📧 Puedes iniciar sesión con tu cuenta de Google en otros lados
- ☁️ Las nubes (AWS, Azure) hacen funcionar el 50% de internet

Sin APIs, cada app sería una isla aislada. Aburrido.

Un ejemplo más real (Weather API)

javascript

```
// Tú pides así:  
https://api.weather.com/forecast?city=Buenos Aires&days=7  
  
// La API responde así:  
{  
  "city": "Buenos Aires",  
  "temp": 25,  
  "condition": "Soleado",  
  "forecast": [...]  
}
```



¿Ves? La API es previsible. Siempre responde en el mismo formato.

Lo que hace especial a una buena API

- ✦ **Clara:** sabes qué pedir y qué esperar
- ✦ **Confiable:** responde siempre igual
- ✦ **Rápida:** no te hace esperar eternidades
- ✦ **Segura:** no cualquiera puede usarla

¿Hay algo específico sobre APIs que quieras explorar más?

     hace 3 minutos

3.2-

Mensaje enviado: Max Length 50, Explica el funcionamiento de una API.

Max Length 50, Explica el funcionamiento de una API.

¿Qué es una API?

Una **API** (Interfaz de Programación de Aplicaciones) es un intermediario que permite que dos programas se comuniquen entre sí.

Funcionamiento básico:

1. **Solicitud:** Tu aplicación envía una petición a la API con datos específicos
2. **Procesamiento:** El servidor procesa tu solicitud
3. **Respuesta:** Devuelve los datos solicitados en un formato estándar (JSON, XML)
4. **Integración:** Tu app utiliza esos datos

Ejemplo simple:

Pedir el clima → API meteorológica recibe tu ubicación → Busca datos → Devuelve temperatura y condiciones → Tu app muestra el resultado

En resumen: Las APIs son como camareros en un restaurante: toman tu pedido (solicitud), van a la cocina (servidor), y traen lo que pediste (respuesta).

📄 🔊 🌟 🔄 🕒 ahora

3.3-

Mensaje enviado: stop sequences 10, explica como funciona una API

Respuesta:

stop sequences 10, explica como funciona una API

Cómo funciona una API

1. **Cliente hace solicitud:** Tu aplicación envía un mensaje a la API (por ejemplo: "dame el clima de Mendoza")
2. **API recibe:** La API recibe tu solicitud en uno de sus "endpoints" (direcciones específicas)
3. **Procesa:** El servidor interpreta qué quieres y busca la información solicitada
4. **Responde:** La API te envía los datos en formato estructurado (generalmente JSON)
5. **Cliente usa datos:** Tu app recibe la respuesta y la muestra al usuario

En resumen: Cliente → Solicitud → Servidor → Procesa → Respuesta → Cliente usa datos

📄 🔊 🌟 🔄 🕒 hace 1 minuto

3.4-

a-

Mensaje enviado: frequency penalty, que es una API?

Respuesta:

frequency penalty, que es una API?

Pensó durante 2s >

¿Qué es una API?

Una **API** es un intermediario que permite que dos aplicaciones se comuniquen y compartan datos.

Lo esencial

- Aplicación de Programación de Interfaz
- Permite que programas hablen entre sí sin que el usuario tenga que hacer nada manual
- Define las reglas de cómo pedir información y cómo recibirla

Ejemplo del mundo real

Cuando compras un vuelo en Google Flights:

- Google no almacena vuelos propios
- Google **usa APIs de aerolíneas** para obtener horarios y precios
- La API aérea devuelve los datos
- Google te muestra las opciones

↓

Básicamente

Una API es un **punto de comunicación** entre programas que permite:

- Solicitar datos
- Enviar información
- Ejecutar acciones
- Todo de forma automática y estructurada

Sin APIs, tendrías que copiar manualmente información de un lugar a otro.

¿Necesitas que profundice en algo específico?



b-

Mensaje enviando: presence penalty, Explica como funciona una API

Respuesta:

presence penalty, Explica como funciona una API

Pensó durante 2s >

Cómo funciona una API

El flujo básico

1. **Solicitud:** Tu aplicación envía una petición a la API con lo que necesitas
2. **Ruta:** La solicitud llega a un endpoint específico (como `/clima` o `/usuarios`)
3. **Procesamiento:** El servidor recibe la solicitud y ejecuta la lógica necesaria
4. **Base de datos:** Busca o modifica datos si es necesario
5. **Respuesta:** El servidor envía los datos de vuelta (normalmente en JSON)
6. **Uso:** Tu aplicación recibe los datos y los muestra

Ejemplo práctico

Pedir el clima:

- Tu app envía: `GET /clima?ciudad=Mendoza`
- API procesa: Busca datos de Mendoza
- API responde: `{temperatura: 28°C, humedad: 45%}`
- Tu app muestra: "28°C en Mendoza"



Métodos principales

- **GET:** Pedir datos
- **POST:** Enviar nuevos datos
- **PUT:** Actualizar datos existentes
- **DELETE:** Eliminar datos

Resumen

API = **Sistema de reglas** que permite a programas comunicarse de forma automática y estructurada.

¿Qué aspecto te interesa más?

     hace 1 minuto

5.3-

Mensaje enviado:

1. Explica Funciones en python.
2. Para explicarlo te vas a convertir en un profesor de programación, y le vas a explicar a tus alumnos de primer año de la universidad.
3. Utiliza la siguiente información para responder: [Tema: Funciones en python (Funciones mas utilizadas y funciones propias)] [Nivel: Principiante.]
4. Explica que son y como funcionan las funciones en python, con un lenguaje sencillo, ejemplos fáciles de entender, enfócate en las ideas principales del tema, y por ultimo quiero un ejemplo donde se pueda aplicar en la vida cotidiana. Max Length 500, Temperatura 0.5

Funciones en Python 🐍

¡Hola estudiantes! Hoy aprenderemos sobre las **funciones**, uno de los conceptos fundamentales en programación.

¿Qué es una función?

Imaginen que una función es como una **receta de cocina**. Cuando ustedes quieren hacer un pastel, siguen los pasos de la receta una y otra vez. En programación, las funciones funcionan igual: son bloques de código reutilizables que realizan una tarea específica.

¿Cómo funcionan?

Las funciones tienen tres partes principales:

1. **Definición:** Se crea con `def`
2. **Ejecución:** Se llama cuando la necesitamos
3. **Retorno:** Devuelve un resultado (opcional)

```
python
def saludar(nombre):
    return f"¡Hola, {nombre}!"

# Llamar la función
mensaje = saludar("María")
print(mensaje) # ¡Hola, María!
```

Respuesta:

Funciones más utilizadas (Built-in)

Python ya tiene funciones predefinidas listas para usar:

- `print()` - Muestra texto
- `len()` - Cuenta elementos
- `sum()` - Suma números
- `max()` - Encuentra el máximo

python

```
números = [5, 10, 3, 8]
print(len(números))    # 4
print(sum(números))    # 26
print(max(números))    # 10
```



Funciones propias (personalizadas)

Ustedes también pueden crear sus propias funciones:

```
python

def calcular_promedio(notas):
    promedio = sum(notas) / len(notas)
    return promedio

mis_notas = [8, 9, 7, 8.5]
resultado = calcular_promedio(mis_notas)
print(resultado) # 8.125
```

Ejemplo de la vida cotidiana 📅

Imaginen que son **meseros en un restaurante**:

- La **función** es: "Tomar el pedido del cliente"
- **Entrada**: Cliente con su menú
- **Proceso**: Anotar lo que pide
- **Salida**: Nota del pedido

python



```
def tomar_pedido(cliente, platillo, cantidad):  
    pedido = f"Cliente: {cliente}\nPlatillo: {platillo}\nCantidad: {cantidad}"  
    return pedido  
  
mi_pedido = tomar_pedido("Juan", "Pizza", 2)  
print(mi_pedido)
```

Ideas principales ✨

- ✓ Las funciones **evitan repetir código**
- ✓ Hacen el código más **organizado y limpio**
- ✓ Son **reutilizables** en cualquier parte del programa
- ✓ Facilitan encontrar y corregir **errores**

Recuerden: Las funciones son como herramientas en una caja de herramientas. Cuando las necesitan, las sacan y las usan una y otra vez. ¡Eso es eficiencia! 🚀